

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# =====
# VOCAB
# =====
CHARS = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
char2idx = {c: i+1 for i, c in enumerate(CHARS)}
idx2char = {i+1: c for i, c in enumerate(CHARS)}
idx2char[0] = ""
NUM_CLASSES = len(CHARS) + 1
```

```
class CaptchaDataset(Dataset):
    def __init__(self, img_dir):
        self.img_dir = img_dir
        self.images = [
            f for f in os.listdir(img_dir)
            if f.lower().endswith(tuple([".png", ".jpg", ".jpeg"]))
        ]

        self.transform = transforms.Compose([
            transforms.Grayscale(),
            transforms.Resize((100, 200)),
            transforms.ToTensor()
        ])

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_name = self.images[idx]
```

```

img_name = self.images[idx]

label = img_name.split("_")[0]

img_path = os.path.join(self.img_dir, img_name)
image = Image.open(img_path).convert("L")
image = self.transform(image)

filtered_label = "".join([c for c in label if c in char2idx])
label_idx = torch.tensor(
    [char2idx[c] for c in filtered_label],
    dtype=torch.long
)

return image, label_idx

```

```

def collate_fn(batch):
    images, labels = zip(*batch)

    images = torch.stack(images)
    label_lengths = torch.tensor([len(l) for l in labels], dtype=torch.long)
    labels = torch.cat(labels)

    input_lengths = torch.full(
        size=(images.size(0),),
        fill_value=7, # Corrected CNN output width from 50 to 7
        dtype=torch.long
    )

    return images, labels, input_lengths, label_lengths

```

```

class CRNN(nn.Module):
    def __init__(self):
        super().__init__()

        resnet = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
        # Modify the first conv layer to accept 1 input channel (grayscale)
        resnet.conv1 = nn.Conv2d(1, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)

        self.cnn = nn.Sequential(*list(resnet.children())[:-2])

        self.conv1x1 = nn.Conv2d(512, 256, 1)

        self.lstm = nn.LSTM(
            input_size=256 * 4,
            hidden_size=256,
            num_layers=2,
            bidirectional=True

```

```

    )

    self.fc = nn.Linear(512, NUM_CLASSES)

    def forward(self, x):
        x = self.cnn(x)          # (B,512,4,50)
        x = self.conv1x1(x)      # (B,256,4,50)

        b, c, h, w = x.size()
        x = x.permute(3, 0, 1, 2).contiguous()
        x = x.view(w, b, c*h)    # (50,B,1024)

        x, _ = self.lstm(x)
        x = self.fc(x)
        return x

```

```

train_ds = CaptchaDataset("/content/drive/MyDrive/precog_captcha/task1/train")
val_ds   = CaptchaDataset("/content/drive/MyDrive/precog_captcha/task1/validation")
test_ds  = CaptchaDataset("/content/drive/MyDrive/precog_captcha/task1/test")

```

```

train_loader = DataLoader(
    train_ds, batch_size=32,
    shuffle=True, collate_fn=collate_fn
)

val_loader = DataLoader(
    val_ds, batch_size=32,
    shuffle=False, collate_fn=collate_fn
)

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = CRNN().to(device)
criterion = nn.CTCLoss(blank=0, zero_infinity=True)
optimizer = optim.Adam(model.parameters(), lr=1e-4)

```

```

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 212MB/s]

```

```

EPOCHS = 30
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for images, labels, in_len, tgt_len in train_loader:

```

```
images = images.to(device)
labels = labels.to(device)

optimizer.zero_grad()
outputs = model(images)
log_probs = outputs.log_softmax(2)

loss = criterion(log_probs, labels, in_len, tgt_len)
loss.backward()
optimizer.step()

total_loss += loss.item()

print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {total_loss/len(train_loader):.4f}")
```

```
Epoch 1/30 | Loss: 2.8981
Epoch 2/30 | Loss: 1.4082
Epoch 3/30 | Loss: 0.9171
Epoch 4/30 | Loss: 0.6308
Epoch 5/30 | Loss: 0.4472
Epoch 6/30 | Loss: 0.3179
Epoch 7/30 | Loss: 0.2097
Epoch 8/30 | Loss: 0.1500
Epoch 9/30 | Loss: 0.1068
Epoch 10/30 | Loss: 0.0802
Epoch 11/30 | Loss: 0.0598
Epoch 12/30 | Loss: 0.0433
Epoch 13/30 | Loss: 0.0338
Epoch 14/30 | Loss: 0.0270
Epoch 15/30 | Loss: 0.0217
Epoch 16/30 | Loss: 0.0179
Epoch 17/30 | Loss: 0.0150
Epoch 18/30 | Loss: 0.0128
Epoch 19/30 | Loss: 0.0110
Epoch 20/30 | Loss: 0.0096
Epoch 21/30 | Loss: 0.0084
Epoch 22/30 | Loss: 0.0074
Epoch 23/30 | Loss: 0.0066
Epoch 24/30 | Loss: 0.0059
Epoch 25/30 | Loss: 0.0054
Epoch 26/30 | Loss: 0.0048
Epoch 27/30 | Loss: 0.0044
Epoch 28/30 | Loss: 0.0040
Epoch 29/30 | Loss: 0.0037
Epoch 30/30 | Loss: 0.0034
```

```
def decode(pred):
    pred = pred.argmax(2)
    prev = -1
    text = ""
```

```

for p in pred:
    p = p.item()
    if p != prev and p != 0:
        text += idx2char[p]
    prev = p
return text

```

```

import editdistance

def char_error_rate(preds, targets):
    total_edits = 0
    total_chars = 0

    for p, t in zip(preds, targets):
        total_edits += editdistance.eval(p, t)
        total_chars += len(t)

    return total_edits / total_chars

```

```

# Redefine decode function with the fix, to ensure it's used correctly within this cell
def decode(pred):
    pred = pred.argmax(1) # Changed from argmax(2) to argmax(1)
    prev = -1
    text = ""

    for p in pred:
        p = p.item()
        if p != prev and p != 0:
            text += idx2char[p]
        prev = p
    return text

model.eval()
all_preds = []
all_targets = []

with torch.no_grad():
    for images, labels, _, label_lengths in val_loader:
        images = images.to(device)
        outputs = model(images)

        for i in range(outputs.size(1)):
            pred_text = decode(outputs[:, i, :])

            target = labels[:, label_lengths[i]]

```

```
target_text = "".join([idx2char[c.item()] for c in target])
```

```
all_preds.append(pred_text)
all_targets.append(target_text)
```

```
char_correct = 0
char_total = 0
word_correct = 0

for p, t in zip(all_preds, all_targets):
    char_correct += sum(pc == tc for pc, tc in zip(p, t))
    char_total += len(t)
    if p == t:
        word_correct += 1

char_accuracy = char_correct / char_total
word_accuracy = word_correct / len(all_targets)
cer = char_error_rate(all_preds, all_targets)

print("Character Accuracy:", char_accuracy)
print("Word Accuracy:", word_accuracy)
print("Character Error Rate (CER):", cer)
```

```
Character Accuracy: 0.2973253700337575
Word Accuracy: 0.1478787878787879
Character Error Rate (CER): 0.7044923396520384
```